

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Sebastián Abarza

12 de septiembre de 2026

02:28

1. Introducción

En este trabajo se presentarán los resultados obtenidos de estudiar el comportamiento experimental de distintos algoritmos de ordenamiento y multiplicación de matrices. Los algoritmos de ordenamiento que serán estudiados en este trabajos son: merge sort, quick sort, patience sort y el algoritmo sort de la librería estandar de C++. Por otro lado, también, los algoritmos de multiplicación de matrices que estudiaremos son: naive y strassen.

El objetivo es medir el tiempo de ejecución y la memoria utilizada por cada algoritmo bajo distintos tamaños de entrada, tipos de datos y dominios de valores, para contrastar el comportamiento observado empíricamente con lo que predice su complejidad teórica

2. Entorno experimental

Sistema operativo: Linux Mint 21.3

CPU: Intel i5-8250U (8) @ 3.400GHz

GPU: Intel UHD Graphics 620 & AMD ATI Radeon R7 M260/M265 / M

RAM: 12 GB

Compilador: g++ (Ubuntu 11.4.0-1ubuntu1 22.04.3) 11.4.0

Se utilizó el dataset generado por el programa `array_generator.py` suministrado para la tarea (en algunos casos, no se utilizaron los arreglos o matrices más grandes debido a problemas de rendimiento).

3. Resultados y Análisis

3.1. Algoritmos de ordenamiento

En la Figura 1 se observa el tiempo de ejecución promedio de cada algoritmo en función del tamaño del arreglo, junto con líneas de referencia teóricas. Los cuatro algoritmos siguen razonablemente bien una tendencia $O(n \log n)$ en la escala logarítmica utilizada, incluyendo Quick Sort, para el cual se utilizó una elección de pivote aleatorio para mejorar su rendimiento. Las pequeñas desviaciones respecto a la curva teórica en cada algoritmo se explican por factores constantes propios de cada implementación así como diferencias en como los algoritmos afrontan distintas entradas, factores que la notación asintótica no captura, pero que sí afectan el tiempo real de ejecución.

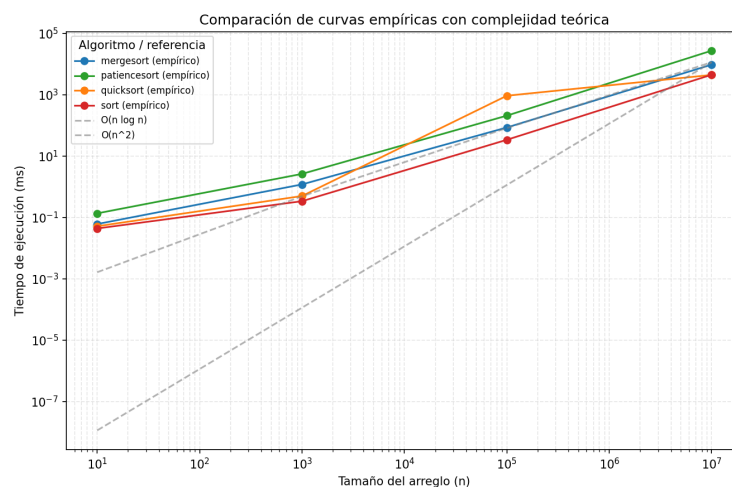


Figura 1: Tamaño del arreglo con respecto al tiempo de ejecución promedio de los casos de prueba. Separado por algoritmo y con complejidad teórica como referencia.

La Figura 2 muestra la memoria auxiliar utilizada por cada algoritmo respecto al tamaño del arreglo. Se observa una clara diferencia según si el algoritmo ordena *in-place* o no: Quick Sort y `std::sort` ambos *in-place*, sin necesidad de estructuras auxiliares en el heap más allá de la pila de recursión muestran un uso de memoria prácticamente nulo y constante, independiente del tamaño de entrada, lo cual coincide con su complejidad de espacio auxiliar $O(\log n)$ (memoria de pila, no medida por este mecanismo basado en el heap). En contraste, Merge Sort y Patience Sort sí requieren memoria auxiliar proporcional a n : Merge Sort porque su función de fusión crea explícitamente vectores temporales para combinar las mitades ordenadas, y Patience Sort porque debe mantener simultáneamente todos los montones (*piles*) generados durante el proceso de inserción. Patience Sort exhibe un uso de memoria notablemente mayor que Merge Sort para un mismo n , lo que se explica por el peor caso estructural del algoritmo: en arreglos ascendentes, cada nuevo elemento es mayor o igual al tope de todos los montones existentes, forzando la creación de un montón nuevo por cada elemento procesado, en vez de reutilizar montones existentes.

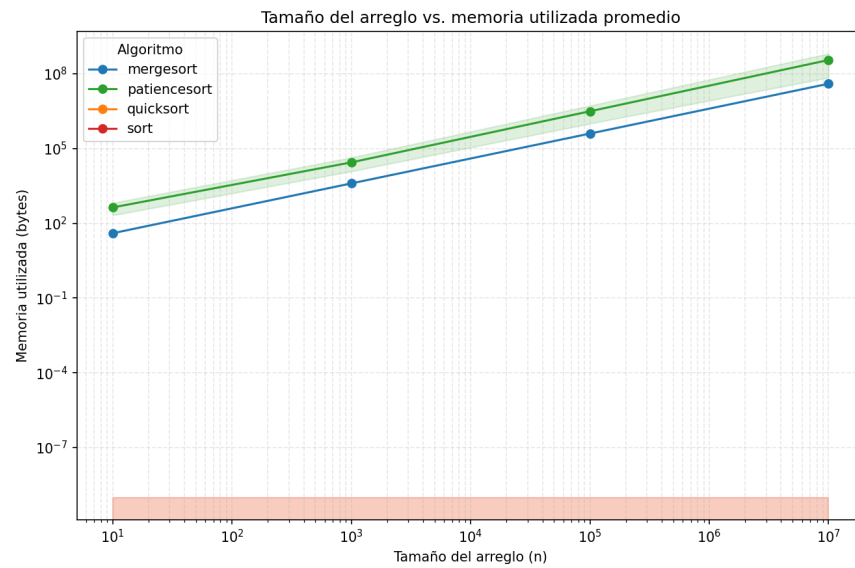


Figura 2: Tamaño del arreglo con respecto a la memoria utilizada promedio de los casos de prueba. Separado por algoritmo.

Finalmente, la Figura 3 compara el tiempo de ejecución promedio de cada algoritmo entre los dominios D1 y D7. Patience Sort es consistentemente el algoritmo más lento de los cuatro, coherente con el mayor overhead de gestión de montones descrito anteriormente.

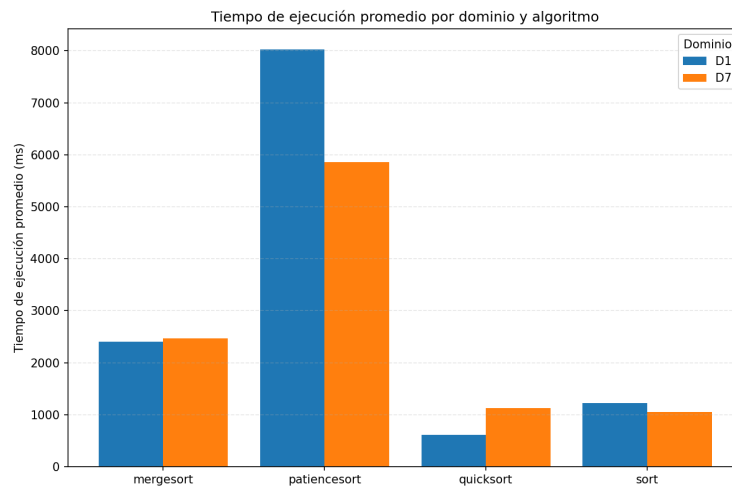


Figura 3: Tiempo de ejecución promedio de cada uno de los algoritmos en los dominios D1 y D7.

3.2. Algoritmos de Multiplicación de Matrices

En cuanto a los algoritmos de multiplicación de matrices, la Figura 4 muestra que tanto el algoritmo naive como Strassen siguen de cerca sus complejidades teóricas esperadas, $O(n^3)$ y $O(n^{2,807})$ respectivamente, en el rango de tamaños evaluado. Esto confirma que ambas implementaciones se comportan según lo predicho por el análisis asintótico, sin degradaciones inesperadas atribuibles a errores de im-

plementación.

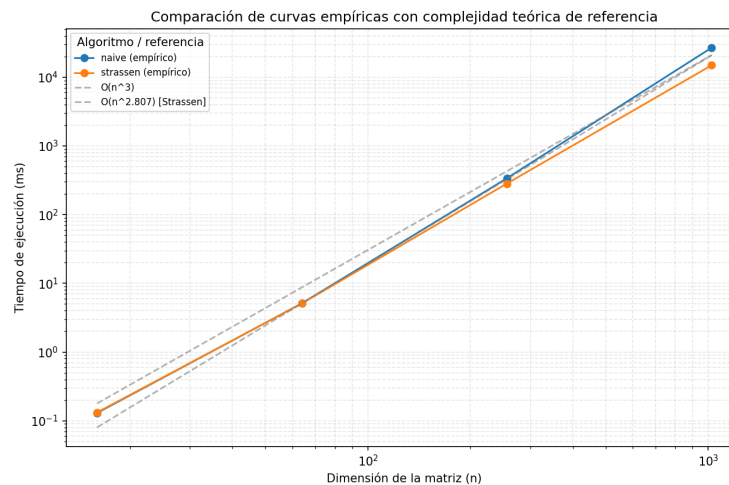


Figura 4: Tamaño de la matriz con respecto al tiempo de ejecución promedio de los casos de prueba. Separado por algoritmo y con complejidad teórica como referencia.

La Figura 5 muestra la memoria auxiliar utilizada por cada algoritmo en función de la dimensión de la matriz. Para tamaños pequeños, ambos algoritmos utilizan una cantidad de memoria similar, ya que Strassen, por debajo de su umbral de caso base, delega directamente en una multiplicación equivalente a la naive sin recurrir a el costo adicional de la recursión. Sin embargo, a partir de cierto tamaño la memoria utilizada por Strassen crece notablemente por sobre la de naive. Esto se explica por la naturaleza recursiva del algoritmo que por cada nivel de la recursión, crea multiples submatrices utilizando memoria adicional.

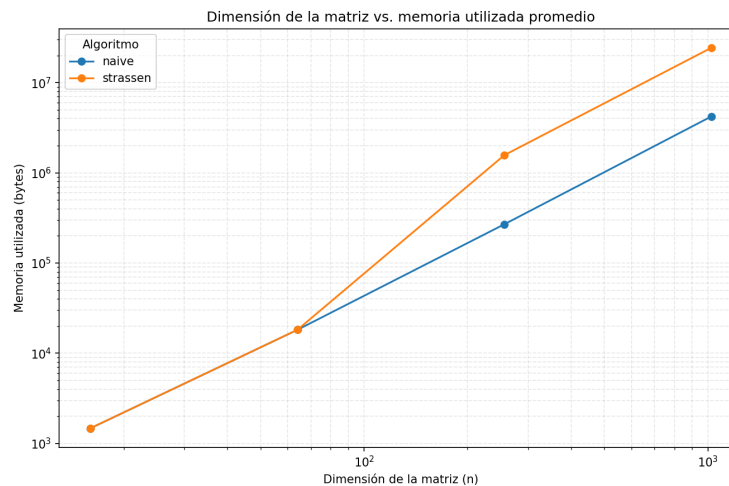


Figura 5: Tamaño de la matriz con respecto a la memoria utilizada promedio de los casos de prueba. Separado por algoritmo.

Por último, la Figura 6 compara el tiempo de ejecución promedio de ambos algoritmos entre los do-

minios D0 ($\{0,1\}$) y D10 ($\{0,\dots,9\}$).

Strassen es notablemente más rápido que naive en ambos dominios, lo que resulta coherente con la Figura 4 y con la ventaja asintótica de Strassen para matrices de mayor tamaño. Además, Strassen muestra un tiempo de ejecución prácticamente idéntico entre D0 y D10, lo cual concuerda con lo esperado. Tanto naive como Strassen realizan exactamente las mismas operaciones aritméticas sin importar los valores concretos de los coeficientes de la matriz, por lo que el dominio de valores no debería afectar el tiempo de ejecución de ninguno de los dos algoritmos, a diferencia de lo observado en algunos algoritmos de ordenamiento donde la cantidad de valores duplicados sí incide en el número de comparaciones realizadas.

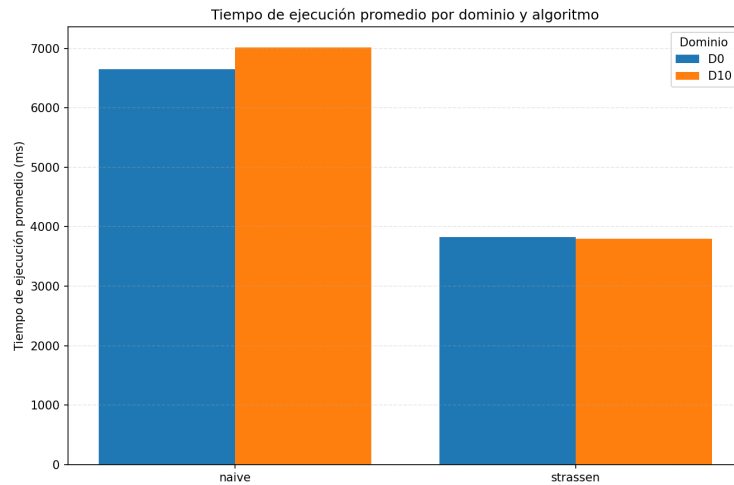


Figura 6: Tiempo de ejecución promedio de cada uno de los algoritmos en los dominios D0 y D10.

4. Conclusiones

Los resultados confirman que la notación asintótica predice correctamente la tendencia general de los algoritmos. Los algoritmos de ordenamiento siguen $O(n \log n)$ (Figura 1), mientras que naive y Strassen se ajustan a $O(n^3)$ y $O(n^{2,807})$ respectivamente (Figura 4). Sin embargo, la complejidad temporal por sí sola no explica todo el comportamiento observado. Por ejemplo, en memoria, Quick Sort y `std::sort` se mantienen prácticamente constantes por ser *in-place*, frente al crecimiento notoriamente mayor de Patience Sort (Figura 2); similarmente, Strassen utiliza notablemente más memoria que naive pese a su ventaja en tiempo (Figuras 6 y 5). Esto evidencia que algoritmos con complejidad temporal similar o mejor pueden diferir considerablemente en su uso de memoria, un trade-off que solo el análisis experimental permite capturar.